

```

1  #include <iostream>
2  #include<bits/stdc++.h>
3  using namespace std;
4  int main(){
5      int n;
6      cin>>n;
7      vector<int>t(n);
8      for(int i=0;i<n;i++) cin>>t[i];
9      int s;
10     cin>>s;
11     bool found=false;
12     for(int i=0;i<n;i++){
13         if(t[i]==s){
14             found=true;
15             break;
16         }
17     }
18     cout<<(found?"Fraud Transaction Found":"Transaction Not Found")<<endl;
19 }

```



```

4
23423
2342
23523
123
1234
Transaction Not Found

...Program finished with exit code 0
Press ENTER to exit console.

```

Fraud Detection (Linear Search)

- Start from first transaction, check one by one
- If match found, print "Fraud Transaction Found" and stop
- If loop ends with no match, print "Transaction Not Found"

```

1  #include <iostream>
2  #include<bits/stdc++.h>
3  using namespace std;
4  int main(){
5      int n;
6      cin>>n;
7      vector<int>v(n);
8      for(int i=0;i<n;i++) cin>>v[i];
9      int t;
10     cin>>t;
11     int lo=0,hi=n-1;
12     bool found=false;
13     while(lo<=hi){
14         int mid=(lo+hi)/2;
15         if(v[mid]==t){found=true;break;}
16         else if(v[mid]<t) lo=mid+1;
17         else hi=mid-1;
18     }
19     cout<<(found?"Log Found":"Log Not Found")<<endl;
20 }

```



```

4
234
341
345
2334
345
Log Found

```

```

...Program finished with exit code 0
Press ENTER to exit console.

```

Server Log Search (Binary Search)

- Since array is sorted, check the middle element
- If target is smaller go left half, if larger go right half
- Keep narrowing until found or range becomes empty

```

1  #include <iostream>
2  #include<bits/stdc++.h>
3  using namespace std;
4  int main(){
5      int n;
6      cin>>n;
7      vector<int>v(n);
8      for(int i=0;i<n;i++) cin>>v[i];
9      for(int i=0;i<n-1;i++)
10         for(int j=0;j<n-1-i;j++)
11             if(v[j]>v[j+1]) swap(v[j],v[j+1]);
12     for(int x:v) cout<<x<<" ";
13 }

```



4

23432

24123

2343

4353

2343 4353 23432 24123

...Program finished with exit code 0
Press ENTER to exit console.

Sort Employee Salaries (Bubble Sort)

Repeatedly compare adjacent elements and swap if left > right
After each full pass, the largest unsorted element "bubbles" to its correct position at the end
Inner loop shrinks by 1 each time since last elements are already sorted

```

1  #include <iostream>
2  #include<bits/stdc++.h>
3  using namespace std;
4  int main(){
5      int n;
6      cin>>n;
7      vector<int>v(n);
8      for(int i=0;i<n;i++) cin>>v[i];
9      for(int i=1;i<n;i++){
10         int key=v[i],j=i-1;
11         while(j>=0&&v[j]>key) v[j+1]=v[j--];
12         v[j+1]=key;
13     }
14     for(int x:v) cout<<x<<" ";
15 }

```

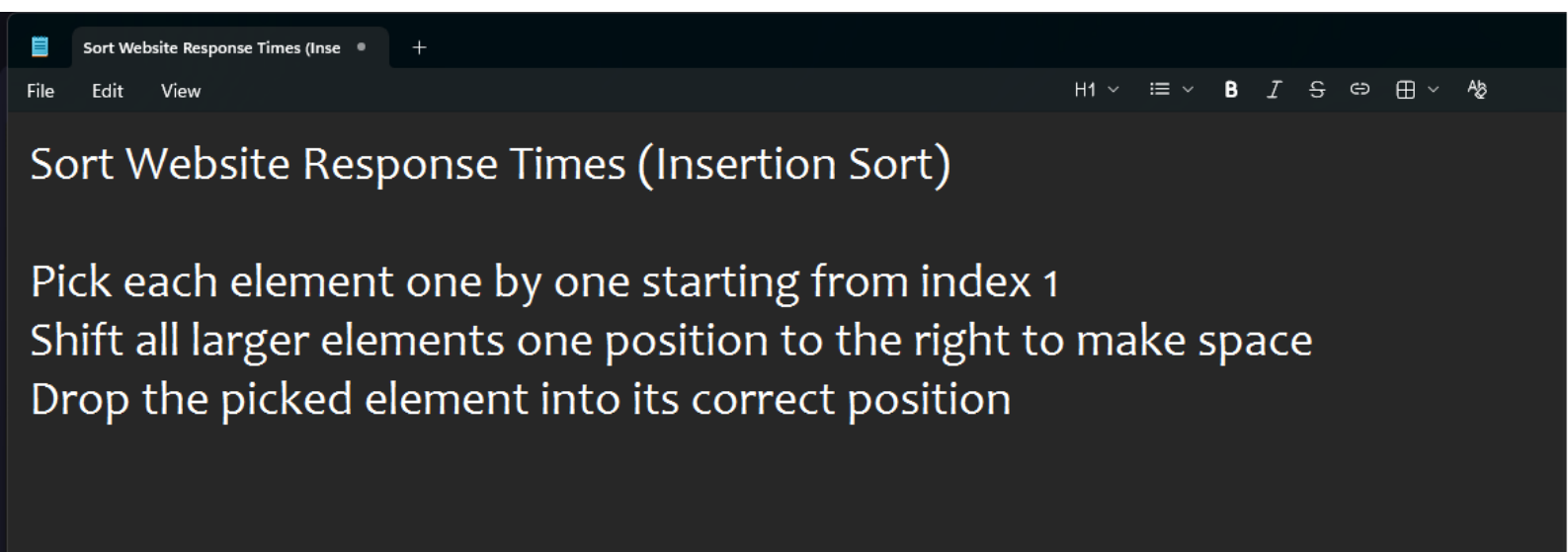


```

4
234
123
4356
34
34 123 4356 34

```

...Program finished with exit code 0
Press ENTER to exit console.




```

1  #include <iostream>
2  #include<bits/stdc++.h>
3  using namespace std;
4  int main(){
5      int n;
6      cin>>n;
7      vector<int>v(n);
8      for(int i=0;i<n;i++) cin>>v[i];
9      for(int i=0;i<n-1;i++){
10         int mi=i;
11         for(int j=i+1;j<n;j++)
12             if(v[j]<v[mi]) mi=j;
13         swap(v[i],v[mi]);
14     }
15     for(int x:v) cout<<x<<" ";
16 }

```



```

4
234
34
245
124
34 124 234 245

```

...Program finished with exit code 0
Press ENTER to exit console.

Prioritize Bug Reports (Selection Sort)

Find the minimum element from the unsorted part
Swap it with the first element of unsorted part
Move the boundary forward by 1 and repeat